

Lineare Modelle für Klassifikation

Contents

- Einfaches Neuronales Netz für Klassifikation
- Logistische Regression
- Aufgaben

Lernziele

- Lineare Neuronale Netze für Klassifikation anpassen: Sigmoid und Softmax
- Cross Entropy Loss als Loss Function für Klassifikation
- Ergebnis: Logistische Regression - das bekannteste lineare Klassifikationsmodell
- Exkurs (nicht prüfungsrelevant): Gradientenabstieg für Logistische Regression

Einfaches Neuronales Netz für Klassifikation

- Nachdem wir ein einfaches NN für Regression eingesetzt haben, wollen wir es für Klassifikation leicht anpassen

Varianten von Klassifikationsmodellen

- **Output:** Zuweisung zu einer Kategorie oder **Wahrscheinlichkeit pro Kategorie** (probabilistische Modelle). Aus der Wahrscheinlichkeit kann man eine Kategorie ableiten, z.B. die mit der höchsten W.keit oder auch mit zusätzlichen Kriterien (Schwellwert)
- **Binary vs. multi-class:** Anzahl Klassen - Zwei Klassen (binary); mehr als zwei Klassen (multi-class)
- Manchmal kann auch mehr als eine Klasse zutreffen, z.B. in einem Zeitungsartikel kann es gleichzeitig um Politik und Umwelt gehen, oder auf einem Bild kann eine Katze und ein Hund sein: **Multi-Label Classification**

Encoding des Outputs

- Es ist nicht sinnvoll, die Klassen als einzelne Zahlen, z.B. 1,2,3, zu encoden; dies suggeriert eine Ordnung zwischen den Klassen, die es i.d.R. nicht gibt
- Stattdessen Repräsentation als Vektoren mit so vielen Einträgen wie es Kategorien gibt, und jeweils einer 1 an der Position, welche die jeweilige Kategorie repräsentiert.
 - Beispiel: Klassifikation von Bildern bzgl. des darauf abgebildeten Tieres, mit den Möglichkeiten "cat", "chicken", und "dog". -> (1, 0, 0) für "cat", (0, 1, 0) für "chicken" und (0, 0, 1) für "dog".
- Dieses Verfahren nennt sich **One-Hot Encoding**

Lineares Modell

- Wir wollen für jeden Feature-Vektor die Wahrscheinlichkeit pro Output-Klasse vorhersagen
- Wir brauchen pro Klasse eine lineare Funktion, wie im Fall der Regression:

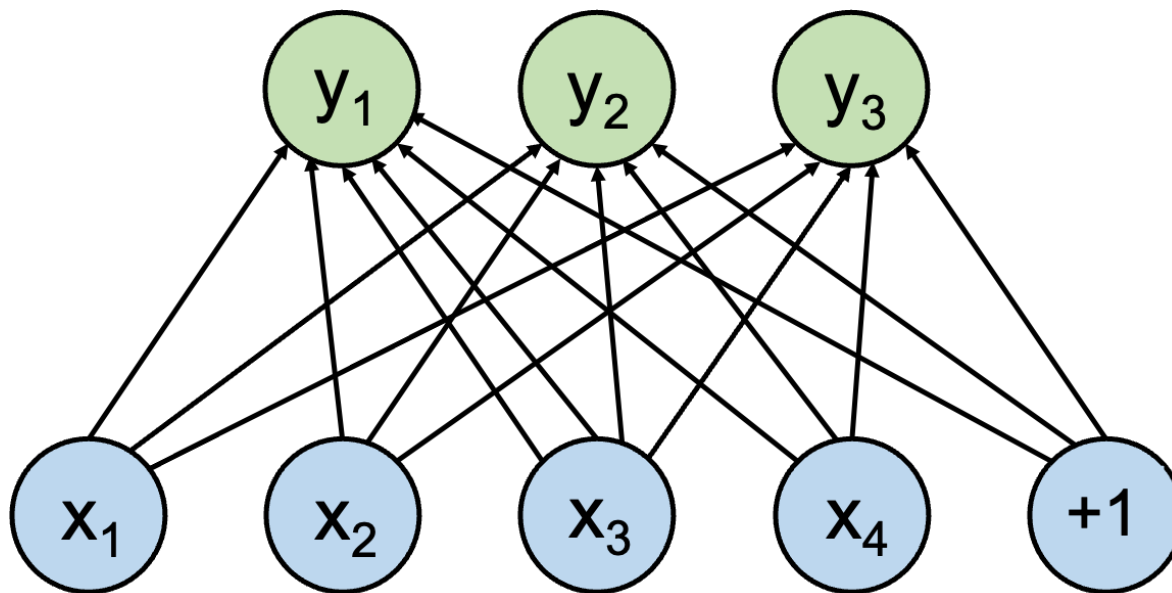
$$y_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + w_{14}x_4 + b_1$$

$$y_2 = w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + w_{24}x_4 + b_2$$

$$y_3 = w_{31}x_1 + w_{32}x_2 + w_{33}x_3 + w_{34}x_4 + b_3$$

- Können wir wieder kompakt in Matrix-Form schreiben: $y = Wx + b$, wobei W eine 3×4 Matrix ist und b ein 3-dim Vektor
- Im Grunde würde eine Gleichung weniger reichen, da Summe der Wahrscheinlichkeiten 1 ergeben muss, aber zur Vereinfachung belassen wir es so
- Insgesamt 15 Parameter zu lernen

Darstellung als NN



Fully connected single layer NN mit 5 Input-Neuronen (einschl. Bias-Term) und 3 Output-Neuronen.

! Übung (3–5 Min.)

Wieso wird dieser Ansatz nicht ganz funktionieren?

✓ Lösung anzeigen

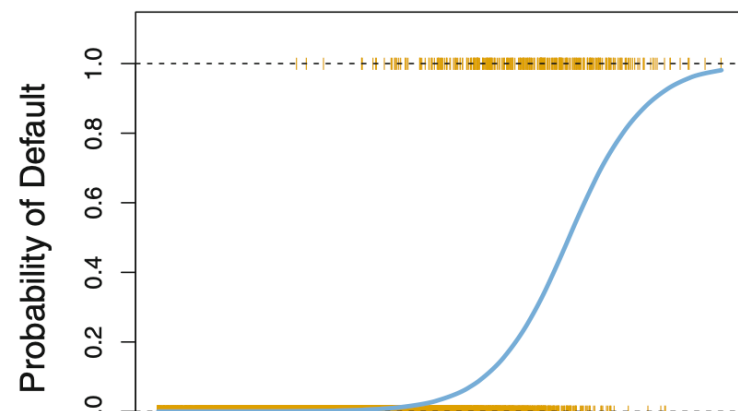
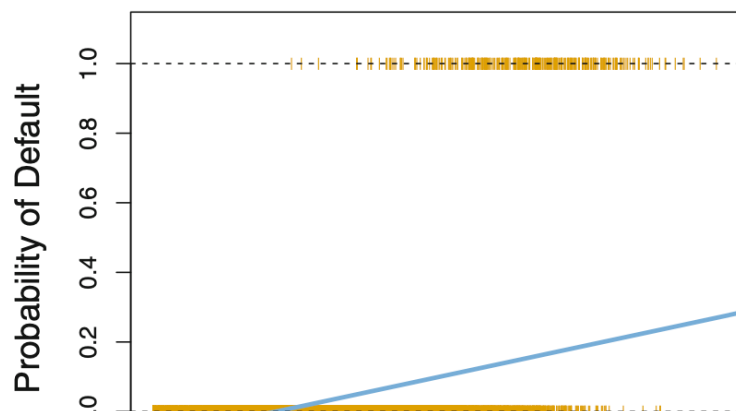
Die Outputs y_i können so beliebige Werte in $\mathbb{R}$ annehmen, aber wir brauchen Werte, die Wahrscheinlichkeiten repräsentieren, insbesondere also:

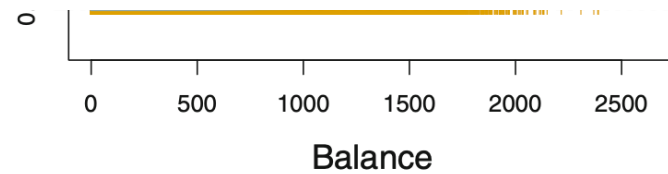
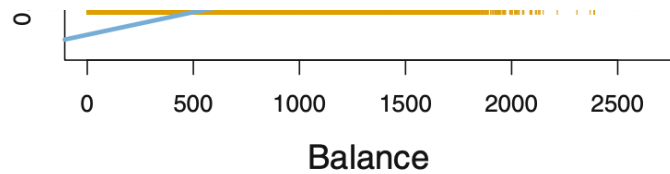
- Werte zwischen 0 und 1,
- deren Summe 1 ist

Wir müssen also das lineare NN $y = Wx + b$ anpassen, damit der Output die gewünschten Eigenschaften hat.

Beispiel mit 2 Klassen

- Datensatz mit Informationen über Kunden einer Kreditkartenfirma. Ziel: vorhersagen, welche Kund*innen mit Kreditkartenschulden in Verzug geraten werden (`default`: Yes / No).
- Nur ein Feature: `balance` (durchschnittliches Guthaben, das Kund*in nach der monatlichen Zahlung noch auf der Kreditkarte hat)





G. James et al., An introduction to statistical learning. p.131. Springer (2017)

Normalisierung des Outputs

Sigmoid-Funktion

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

- bildet $\mathbb{R}$ auf Intervall $(0, 1)$ ab
- Funktion fast linear um $x = 0$ aber flacht stark ab für besonders große bzw. kleine Werte. Dadurch werden Ausreisser gut reguliert (x -Werte, für welche die lineare Funktion besonders große/kleine $\hat{y}$ produziert)
- Sie ist differenzierbar (wichtig für das Lösen der Optimierungsaufgabe beim Training)

Wir brauchen noch, dass die Wahrscheinlichkeiten für beide Klassen in Summe 1 ergeben:

$$\Pr(y = 1|x) + \Pr(y = 0|x) = 1$$

Das erreichen wir durch folgende Definition:

$$\Pr(y = 1|x) = \sigma(w \cdot x + b) = \frac{1}{1 + \exp^{-(w \cdot x + b)}}$$

$$\Pr(y = 0|x) = 1 - \Pr(y = 1|x) = 1 - \frac{1}{1 + \exp^{-(w \cdot x + b)}}$$

Softmax-Funktion

Bei mehr als 2 Klassen muss die Sigmoid-Funktion verallgemeinert werden:

$$\hat{y} = (\hat{y}_1, \dots, \hat{y}_q) = \text{softmax}(o) = (\text{softmax}(o_1), \dots, \text{softmax}(o_q)),$$

wobei

$$\text{softmax}(o_i) := \frac{\exp(o_i)}{\sum_{j=1}^q \exp(o_j)}$$

- Softmax dient im Output, ebenso wie Sigmoid, der **Normalisierung**
- höchster Eintrag in $\hat{y}$ entspricht der wahrscheinlichsten Klasse

Vektorisierung

- Für kompaktere Schreibweise und effiziente Berechnung vektorisieren wir wieder die Berechnung:
- Angenommen, wir haben einen Batch Daten $X \in \mathbb{R}^{n \times d}$, d.h. n Datenbeispiele mit jeweils d Features.
- Angenommen wir haben q Kategorien im Output
- Dann gilt für die Gewichte-Matrix W und den Bias-Vektor b : $W \in \mathbb{R}^{d \times q}$, $b \in \mathbb{R}^{1 \times q}$.

$$O = X \cdot W + b$$
$$\hat{Y} = \text{softmax}(O)$$

- Die Softmax-Funktion wird zeilenweise berechnet (jede Zeile ein Datenbeispiel)

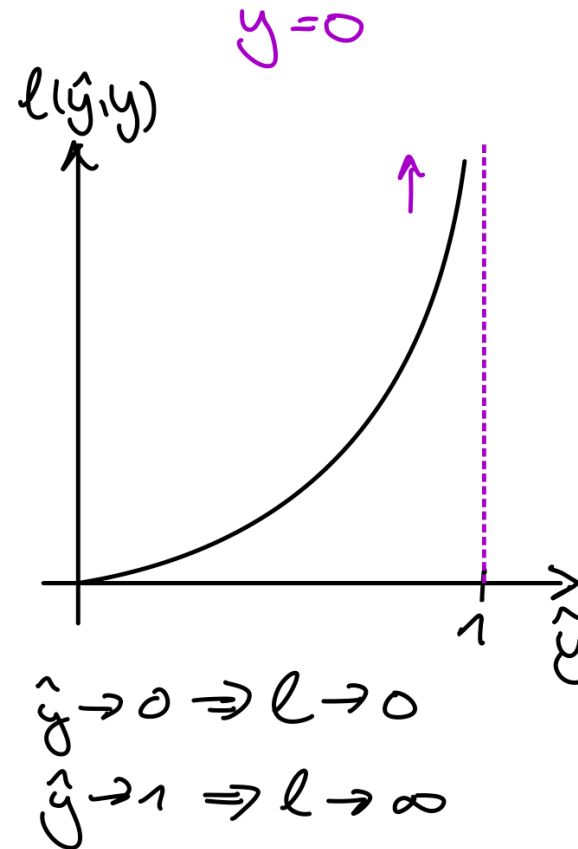
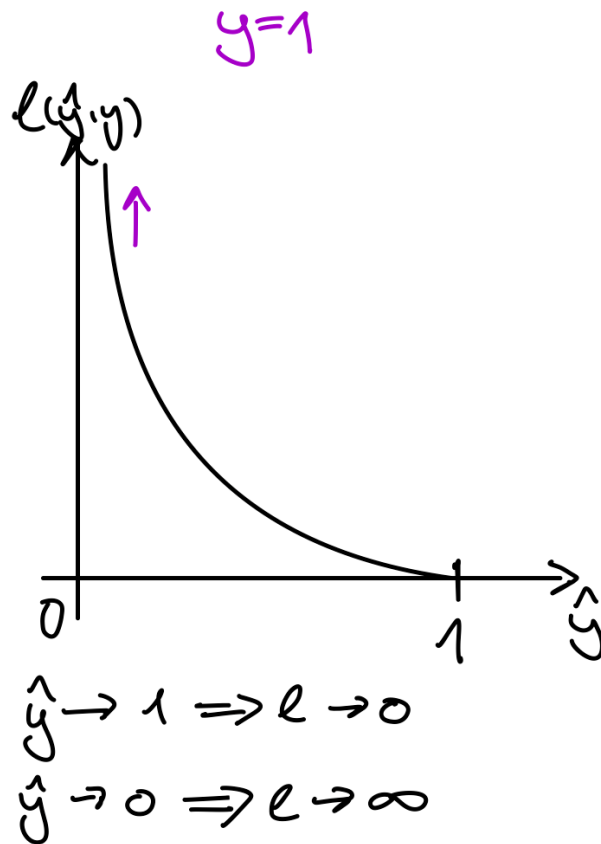
Loss Function

- Ungenauigkeit wäre naheliegend aber ist aus analytischen Gründen nicht gut geeignet für das Lösen der Optimierungsaufgabe im Training
- Für das Herleiten einer passenderen Loss Function betrachten wir einen 1-dim. Feature-Raum und binäre Klassifikation (möglicher Output: 1 oder 0)

$$l(\hat{y}, y) = \begin{cases} -\log(\hat{y}) & \text{für } y = 1 \\ -\log(1 - \hat{y}) & \text{für } y = 0 \end{cases}$$

wobei $0 < \hat{y} < 1$ und $y \in \{0, 1\}$.

Graphische Interpretation:



- Diese Loss Funktion heißt **Cross Entropy Loss**
- man kann sie auch folgendermaßen schreiben (denn einer der beiden Terme ist immer gleich 0):

$$l(\hat{y}, y) = -y \cdot \log(\hat{y}) - (1 - y) \cdot \log(1 - \hat{y})$$

Im Allgemeinen, also für q Klassen, ist die Cross Entropy Loss l folgendermaßen definiert: Für ein Datenbeispiel mit

realem Output y und vorhergesagtem Output $\hat{y}$ gilt

$$l(\hat{y}, y) = - \sum_{i=1}^q y_i \cdot \log(\hat{y}_i)$$

- wieder: alle Terme bis auf einen $= 0$, denn y ist ein one-hot vector (der Länge q)
- l ist nach unten durch 0 beschränkt (da $\hat{y}$ zwischen 0 und 1, und nie 1 wegen softmax)

Logistische Regression

- Das lineare NN entspricht der sogenannten **Logistischen Regression**
- Es ist eines der wichtigsten linearen Modelle für Klassifikation [^1]
- Als mathematische Funktion ist die logistische Regression nicht linear aber als ML Modell schon, denn im ML geht es um die Gleichung, welche die Entscheidungsgrenze (decision boundary) beschreibt, und diese ist linear:

$$wx + b = 0$$

- Die Entscheidungsgrenze ist eine sog. **Hyperebene**
- Im Gegensatz zur linearen Regression hat die logistische Regression keine geschlossene Lösung. Das Auffinden des optimalen Modells benötigt ein Verfahren wie Gradientenabstieg

[^1] Achtung: das Wort "Regression" in "Logistische Regression" kommt daher, dass es von der Linearen Regression abgeleitet ist. Es handelt sich um ein Klassifikationsmodell.

Interpretation der Gewichte

$$\ln \frac{P(y = 1|x)}{P(y = 0|x)} = w \cdot x + b$$

- Positives Gewicht w_i : Feature erhöht W.keit für Klasse 1
- Negatives Gewicht w_i : Feature reduziert W.keit für Klasse 1
- $|w_i|$: Stärke des Effekts

Das unterscheidet Logistische Regression (und Lineare Regression) von fast allen anderen ML-Modellen

Vor- und Nachteile

Vorteile

- schnell, robust, interpretierbar
- gute Baseline
- arbeitet gut mit vielen Samples
- liefert Wahrscheinlichkeiten
- wenig Overfitting (bei passenden Dimensionen)

Nachteile

- linearer Klassifikator → keine komplexen Grenzen
- empfindlich gegen stark korrelierte Features
- funktioniert schlecht bei nicht-linear separierbaren Klassen

Exkurs: Gradienten der Cross-Entropy Loss für binäre Klassifikation

Dieser Abschnitt ist nicht prüfungsrelevant

Gegeben sei ein binärer Klassifikator mit Modelloutput

$$\hat{y} = \sigma(z(x)), \quad z(x) = wx + b, \quad \sigma(z) = \frac{1}{e^{-z}}$$

Unser Ziel ist es, die partiellen Ableitungen der binären Cross-Entropy-Loss

$$l = - (y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y}))$$

nach den Gewichten w_i sowie dem Bias b zu berechnen.

Aus der Analysis nutzen wir drei wichtige Werkzeuge:

1. Ableiten ist linear, d.h. die Ableitung einer Summe von Funktionen ist gleich der Summe der Ableitungen
2. Die Kettenregel
3. die Ableitung des Logarithmus: $\ln'(x) = 1/x$

Demnach gilt:

$$\frac{\partial l}{\partial w_j} = \frac{\partial l}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_j}$$

und

$$\frac{\partial l}{\partial b} = \frac{\partial l}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial b}$$

1. Ableitung von l nach $\hat{y}$

Wir betrachten zuerst den ersten Ausdruck im Produkt auf der rechten Seite:

$$\begin{aligned}\frac{\partial}{\partial \hat{y}} l &= \frac{\partial}{\partial \hat{y}} (-y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})) \\ &= -\frac{\partial}{\partial \hat{y}} y \ln(\hat{y}) - \frac{\partial}{\partial \hat{y}} (1 - y) \ln(1 - \hat{y}) \\ &= -\left(\frac{y}{\hat{y}} - \frac{1 - y}{1 - \hat{y}} \right) \\ &= \frac{\hat{y} - y}{\hat{y} \cdot (1 - \hat{y})}\end{aligned}$$

2. Ableitung von $\hat{y}$ nach z

Da $\hat{y} = \sigma(z)$ mit

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

gilt:

$$\frac{\partial}{\partial z} \hat{y} = \hat{y} \cdot (1 - \hat{y})$$

3. Ableitung von z nach w_j bzw. b

Da $z = wx + b = w_1x_1 + \dots + w_nx_n + b$, ergibt sich

$$\frac{\partial z}{\partial w_j} = x_j$$

und

$$\frac{\partial z}{\partial b} = 1$$

4. Zusammensetzen

$$\begin{aligned}\frac{\partial l}{\partial w_j} &= \frac{\partial l}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_j} \\ &= \frac{\hat{y} - y}{\hat{y} \cdot (1 - \hat{y})} \cdot \hat{y} \cdot (1 - \hat{y}) \cdot x_j \\ &= (\hat{y} - y) \cdot x_j\end{aligned}$$

sowie

$$\begin{aligned}\frac{\partial l}{\partial b} &= \frac{\partial l}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial b} \\ &= \frac{\hat{y} - y}{\hat{y} \cdot (1 - \hat{y})} \cdot \hat{y} \cdot (1 - \hat{y}) \\ &= \hat{y} - y\end{aligned}$$

Geometrische Interpretation

- Der Term $(\hat{y} - y)$ ist positiv, wenn das Modell zu dem Zeitpunkt eine zu hohe W.keit für Klasse 1 vorhersagt und negativ, wenn die W.keit für Klasse 1 zu niedrig ist
- Dieser Ausdruck wird im Gradienten an den Datenpunkt x multipliziert und bestimmt die Richtung und Stärke der Verschiebung
- Geometrisch bedeutet das: der Datenpunkt x zieht an der Entscheidungsgrenze; je größer der Fehler, desto stärker zieht er
- Korrekt klassifizierte Punkte weit weg von der Entscheidungsgrenze haben kaum Einfluss -> stabiler Lernprozess
- Falsch klassifizierte Punkte weit weg von der Entscheidungsgrenze haben großen Einfluss auf den Gradienten -> führt zu großen Korrekturen, damit der Punkt auf die richtige Seite rutscht

Aufgaben

Aufgabe 1 - Logistische Regression interpretieren

Angenommen, wir haben einen Datensatz an Dokumenten, für welche wir folgende Features extrahiert haben und für welche eine logistische Regression mit folgenden Gewichten und Bias $b = 0.1$ trainiert worden ist, um das Sentiment des Textes vorherzusagen:

Feature	Feature Beschreibung	Gelerntes Gewicht
x_1	Zahl positiver Wörter im Dokument	2.5
x_2	Zahl negativer Wörter im Dokument	-5
x_3	1, falls "no" im Dokument, sonst 0	-1.2
x_4	Anzahl Pronomen in 1. und 2. Person	0.5
x_5	1, falls "!" im Dokument, sonst 0	2.0
x_6	$\log(\text{Anzahl an Wörtern im Dokument})$	0.7

Positive Wörter sind so was wie `great`, `nice`, etc.

- a) Was lernen Sie aus den Gewichten über das Modell? Wie wichtig ist welches Feature und welchen Beitrag leistet es?
- b) Berechnen Sie die Wahrscheinlichkeiten, dass das Sentiment für das Dokument in der unteren Abbildung als positiv ($P(y = 1)$) bzw. negativ ($P(y = -1)$) vorhergesagt wird. Leiten Sie daraus ab, zu welcher Klasse das Dokument zugewiesen werden würde.
- c) Berechnen Sie den Wert der Loss Function (Cross Entropy Loss) für das Beispiel aus der Abbildung, einmal unter der Annahme, dass der korrekte Wert der Output Variable $y = 1$ ist, und einmal unter der Annahme, $y = 0$. Überzeugen Sie sich von der Richtigkeit der grundlegenden Logik der Cross Entropy Loss für dieses Beispiel.

It's **hokey**. There are virtually **no** surprises, and the writing is **second-rate**.
 So why was it so **enjoyable**? For one thing, the cast is

$x_2=2$
 $x_3=1$

So why was it so **enjoyable**? For one thing, the cast is

great. Another **nice** touch is the music. **I** was overcome with the urge to get off the couch and start dancing. It sucked **me** in, and it'll do the same to **you**.

$$x_1=3$$

$$x_5=0$$

$$x_6=4.19$$

$$x_4=3$$

Aufgabe 2 - Logistische Regression implementieren

Trainieren Sie eine Logistische Regression anhand des Palmer Penguins Datensatzes. Vergleichen Sie die Ergebnisse mit den Ihres k -NN Modells. Welches performt besser?